



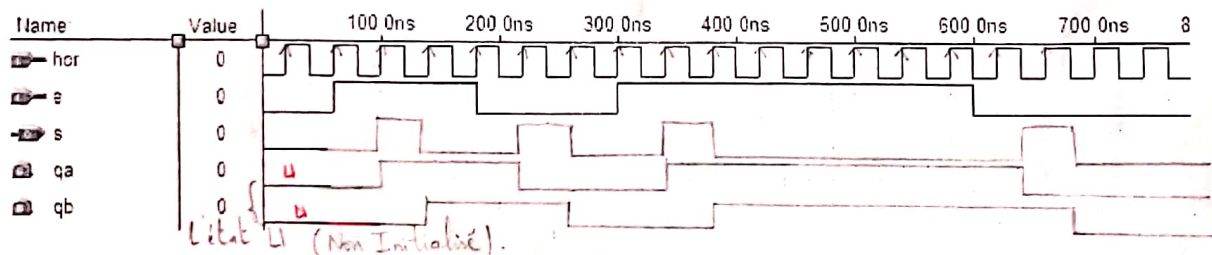
TD 1 : Modélisation par VHDL

Exercice 1 :

On considère le code VHDL suivant :

<pre>LIBRARY IEEE; USE IEEE.std_logic_1164.all; ENTITY transitm IS PORT(hor, e : IN STD_LOGIC; s : out STD_LOGIC); END ; ARCHITECTURE arch OF transitm IS SIGNAL qa : STD_LOGIC; SIGNAL qb : STD_LOGIC;</pre>	<pre>BEGIN S <= qa xor qb; Process (hor) begin if rising_edge (hor) then qa <= e; qb <= qa; end if; end process; END arch;</pre>
---	---

1. Représenter ce code sous forme d'un schéma ?
2. Complétez les chronogrammes suivants ?



Exercice 2 : Modélisation d'un compteur

Entité:

- Signaux d'entrée : CLK (horloge de type bit), RST (reset de type bit)
- Signal de sortie : COUNT (valeur du compteur de type integer)

Architecture :

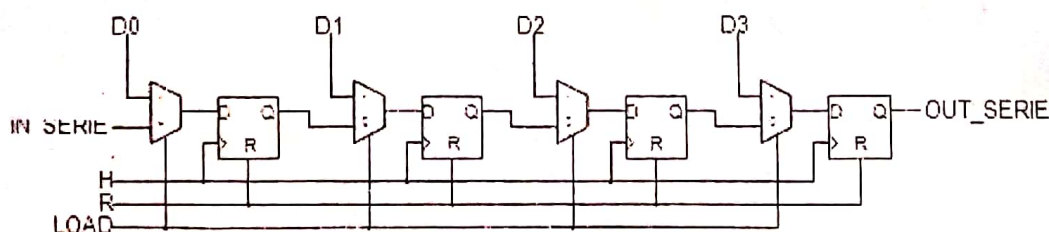
si reset actif alors la valeur du compteur = 0
sinon à chaque flanc montant de l'horloge:
si la valeur du compteur > MAX alors la valeur du compteur = 0
sinon la valeur du compteur = valeur du compteur + 1

NB : Le reset est actif au niveau haut (RST = '1') et la valeur MAX peut être choisie librement

1. Donner le code VHDL de ce compteur ?
2. Modifier ce code pour que l'entrée RST soit synchrone ?

Exercice 3 :

1. Donner le code VHDL des bascules RS et JK ?
2. Donner le code VHDL d'un compteur/décompteur synchrone de 4 bits ?
3. Donner le code VHDL d'un compteur synchrone à prépositionnement synchrone ?
4. Donner le code VHDL d'un registre à décalage à droite ?
5. Donner le code VHDL d'un registre à décalage à droite à chargement parallèle

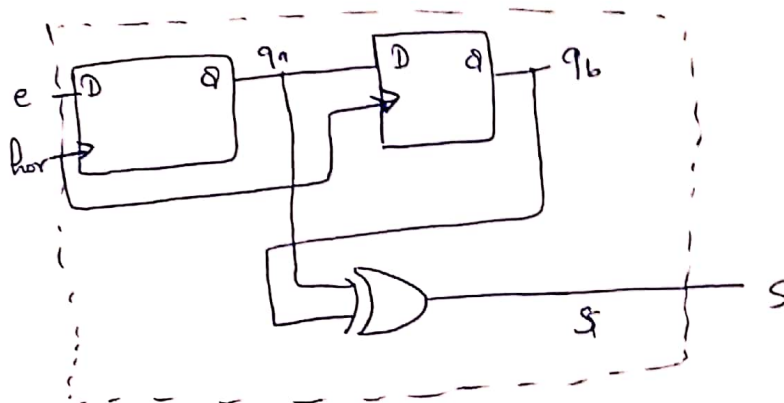
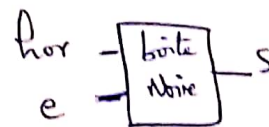


Exo 1:

TD 1

1) nb d'instructions = nb de bascules

$$\left. \begin{array}{l} q_a \leq e \\ q_b \leq q_a \end{array} \right\} 2 \text{ bascules}$$



2) Pour éviter l'état U:

```
process (hor, Rst)
begin
  if (Rst = '0') then
    q_a <= '0';
    q_b <= '0';
```

$q_b \leq q_a$ mais après un décalage d'horloge.

Exo 2:

le VHDL n'accepte pas d'utiliser une sortie comme entrée ($\bar{x}: i = i+1$)

Par bit ou ^{st logic} integer '0' mais pour integer just 0.

* Solution:

```
Entity Compteur is port (clk, Rst: in bit;
                          Count: out integer);
```

```
end;
```

```
Architecture Arch of Compteur is
```

```
  signal Max: natural := 9;
```

```
  signal x: integer;
```

```
begin
```

```
  process (clk, Rst)
```

begin

if (Rst = '1') then $X \leq 0$;

elsif (clk 'event' and clk = '1') then

if ($X > \text{Max}$) then

$X \leq 0$;

else

$X \leq X + 1$;

endif;

endif;

end process;

Count $\leq X$;

end Arch;

(Pour résoudre le prob du sortie $\leq$ sortie).

d) il faut juste process (clk, Rst).

Exercice 3:

1)	R	S	Q_n
	0	0	Q_{n-1}
	0	1	1
	1	0	0
	1	1	X

Entity bascule_RS is port (R, S : in std_logic;
Q : out std_logic);

end;

Architecture Arch of bascule_RS is

signal X : std_logic;

begin

$X \leq '0'$ when R = '1' and S = '0' else

'1' when R = '0' and S = '1' else

X when R = '0' and S = '0' else

'-' ;

$Q \leq X$;

end Arch;

Suite Exo 3:

Entity bascule-jk is port
 (clk, J, k : in std_logic;
 Q : out std_logic);

end;

Architecture Arch of bascule-jk is

Signal X : std_logic;

begin

process (clk)

begin

if (clk 'event' and clk = '1') then

if (J = '1' and k = '0') then X <= '1';

elsif (J = '0' and k = '1') then X <= '0';

elsif (J = '0' and k = '0') then X <= X;

else X <= not (X);

endif

endif

end process

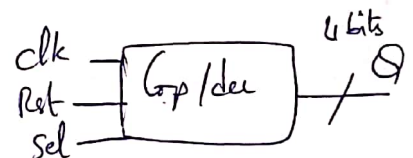
Q <= X;

end Arch;

J	k	Q
0	0	Q _{n-1}
0	1	0
1	0	1
1	1	Q _{n-1}

if et case we use
 then just à l'intérieur
 du process

Reset in general asynchrone



2) Entity Compteur-dec is port (clk, Rst, sel : in std_logic;

Q : out std_logic_vector (3 down to 0));

end;

Architecture Arch of Compteur-dec is

signal X : std_logic_vector (3 down to 0);

begin

process (clk, Rst)

begin

if (Rst = '1') then X <= "0000";

elsif (clk 'event' and clk = '1') then

if (sel = '1') then X <= X + 1;

else X <= X - 1;

endif;

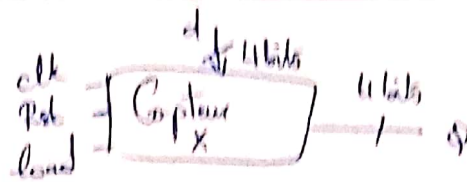
endif;

end process;

Q <= X;

X 
 ← registre de 4 bits

3)



Signal X : std logic-vector (3 down to 0);

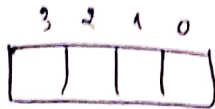
```

begin
  process (clk, Rst)
  begin
    if (Rst = '1') then X <= "0000";
    elsif (clk' ... ) then
      if (load = '1') then X <= d;
      else X <= X + 1;
    end if;
  end if;
end process;

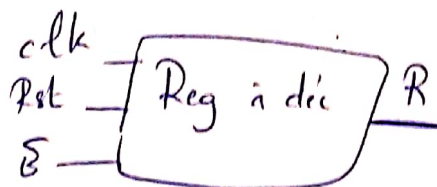
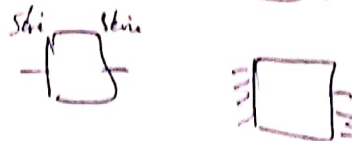
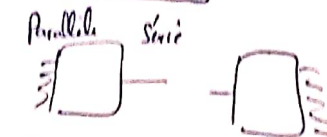
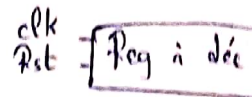
Q <= X;
end Arch;

```

4)



les convertisseurs :



4)

Architecture Arch of reg-dec is

Signal X : std_logic_vector (3 downto 0);

begin

process (clk, Rst)

begin

if (Rst = '1') then

X <= "0000";

elsif (clk'event and clk = '1') then

X <= E & X (3 downto 1);

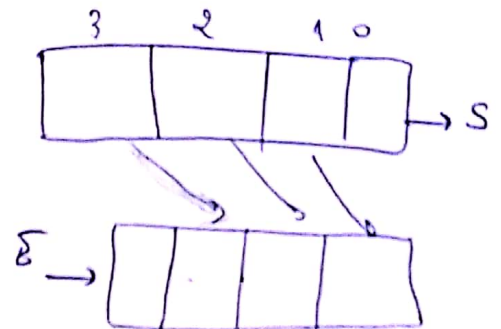
end if

end if

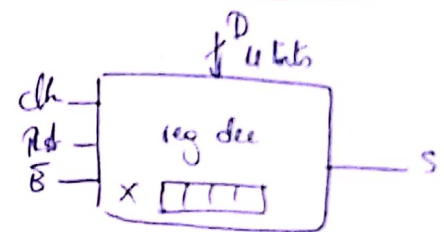
end process;

S <= X(0);

end Arch;



load is synchrone



5)

Architecture Arch of reg-dec-par is

Signal X : std_logic_vector (3 downto 0);

begin

process (clk, Rst)

begin

if (Rst = '1') then

X <= "0000";

elsif (clk'event and clk = '1') then

if (load = '1') then X <= D;

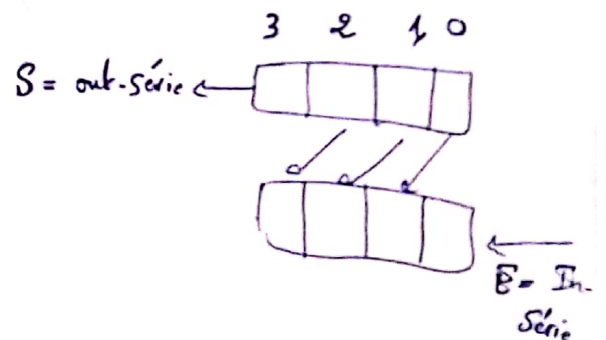
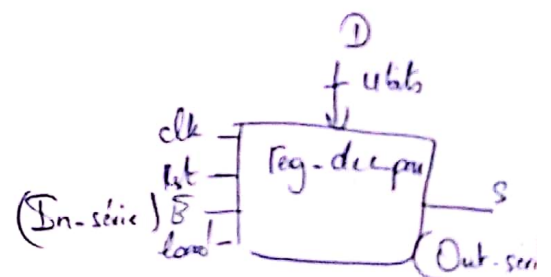
else X <= X (2 downto 0) & E;

end if;

end if;

end process;

S <= X(3); end Arch;





TD 2 : Conception par VHDL

Exercice 1 :

1. Dessinez à partir d'éléments de base le schéma correspondant à ce code *vhdl* :

```
Entity circuit is port (x1, x2, x3, sel : in std_logic;  
                        y : out std_logic);  
End;
```

```
Architecture arch of circuit is  
signal a, b, c, e, f: std_logic;  
begin
```

```
    a <= x1 and x2;
```

```
    b <= x1 xor x2;
```

```
    c <= x3 and b;
```

```
    e <= a or c;
```

```
    f <= x3 xor b;
```

```
    process (e, f, sel)
```

```
    begin
```

```
        if sel='0' then y <= e;
```

```
        else y <= f;
```

```
        end if;
```

```
    end process;
```

```
End architecture;
```

process combinatoire (pas de clk).

2. Quel est la fonction réaliser par ce circuit ?

Exercice 2 :

Dessinez le schéma, après la synthèse logique, du code *vhdl* suivant :

```
LIBRARY IEEE;  
USE IEEE.std_logic_1164.all;  
  
Entity toto is  
port ( A, B, C : in std_logic;  
       Q1, Q2 : out std_logic );  
end;
```

```
architecture synth of toto is
```

```
signal V, R : std_logic;
```

```
begin
```

```
    R <= B xor C;
```

```
process (A)
```

```
begin
```

```
    if (A' event and A='1') then
```

```
        if (V = '1') then Q2 <= C;
```

```
        else Q2 <= B;
```

```
        end if;
```

1 bascule

```
    end if;
```

```
end process;
```

```
process (A)
```

```
begin
```

```
    if (A' event and A='1') then
```

```
        Q1 <= C;
```

```
        V <= R;
```

```
    end if;
```

```
end process;
```

```
end synth;
```

2 bascules

Exercice 3 : Comparateur

On veut modéliser un circuit permettant de comparer 2 bus de données de 8 bits, A et B, et de générer 3 signaux de sortie :

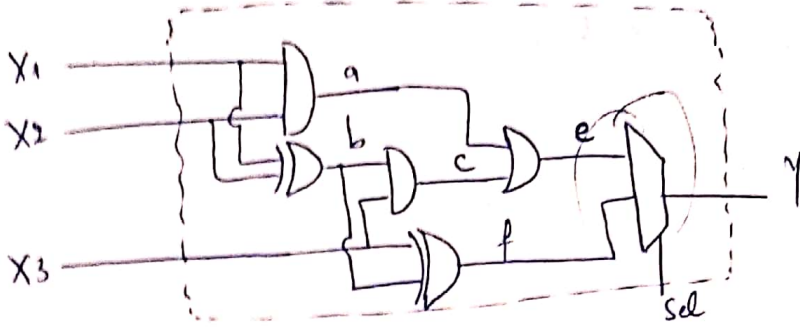
- EQUA si les données sur les deux bus sont égales,
- SUPE si la donnée du bus A est supérieure à la donnée du bus B,
- INFE si la donnée du bus A est inférieure à la donnée du bus B,

Exercice 1.

TD 2

1)

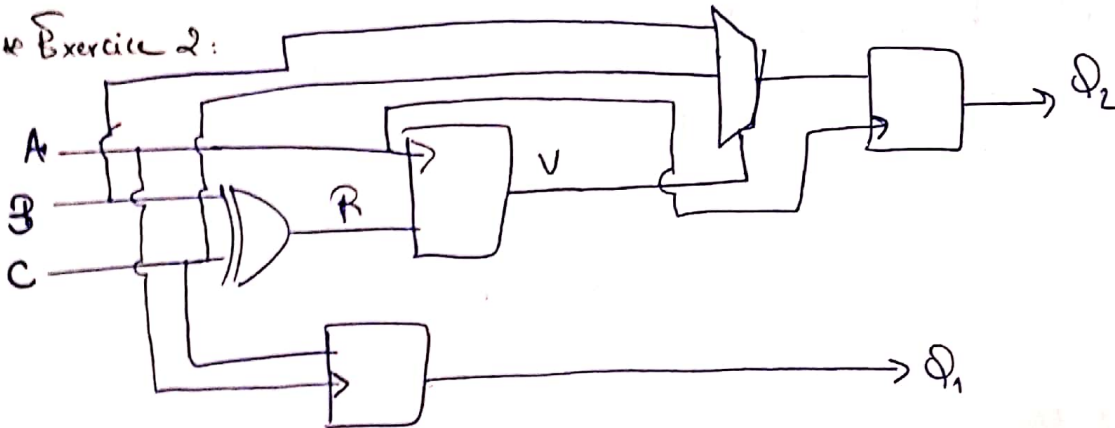
Circuit



$y \leq e$ when $sel = '0'$
else f ;

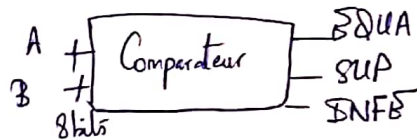
2) C'est un multiplexeur, purement combinatoire.

Exercice 2:



Exercice 3:

Architecture Arch of Comparsateur is



begin

$EQUA = '1'$ when $A = B$ else $'0'$;

$SUP = '1'$ when $A > B$ else $'0'$;

$INFE = '1'$ when $A < B$ else $'0'$;

end Arch;

Exercice 4 : Détecteur de parité

On souhaite modéliser un détecteur de parité pour des mots de 4 bits par une description VHDL :

- ✓ Interface (entité) :
 - Signal d'entrée (mode in) : DIN (données de 4 bits)
 - Signaux de sortie (mode out): ODP (parité impaire d'un bit), EVP (parité paire d'un bit)
- ✓ Comportement (architecture) :
 - ODP= '1' si le nombre de '1' dans DIN est impaire, sinon ODP= '0'
 - EVP='1' si le nombre de '1' dans DIN est pair, sinon EVP= '0'

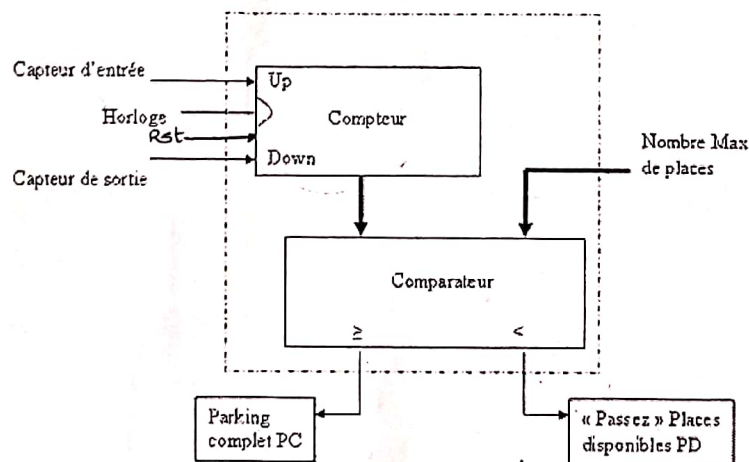
- 1) Donnez la table de vérité selon les différentes valeurs possibles de DIN.
- 2) Codez le comportement avec l'instruction d'assignation sélective (with.... Select).

Exercice 5 : Conception d'un système de gestion de parking

Le but de notre système est de gérer la disponibilité des places d'un parking. Ce système possède 4 signaux d'entrée, les captures d'entrée-sortie, l'horloge et le nombre maximal de places dans le parking (mot binaire de 4 bits). Les entrées capteurs servent à incrémenter ou à décrémenter le compteur (4 bits). La sortie de compteur est comparée au nombre maximal de places dans le parking.

Les signaux de sortie sont :

- ✓ PD = 1 s'il reste des places disponibles si non PD=0.
- ✓ PC = 1 si le parking est complet si non PC=0.



Donnez le code vhdl de ce système.

Exercise 4:

1)



DIN				ODP input	EVP par
0	0	0	0	0	1
0	0	0	1	1	0
0	0	1	0	1	0
0	0	1	1	0	1
0	1	0	0	1	0
0	1	0	1	0	1
0	1	1	0	0	1
0	1	1	1	1	0
1	0	0	0	1	0
1	0	0	1	0	1
1	0	1	0	0	1
1	0	1	1	1	0
1	1	0	0	0	1
1	1	0	1	1	0
1	1	1	0	1	0
1	1	1	1	0	1

2) Entity parity is port (DIN : In std-logic-vector (3 downto 0));
 ODP, EVP : out std-logic);

end;

Architecture Arch of parity is

signal x : std-logic;

begin

with DIN select

$ODP \leq '1'$ when "0001" or "0010" or "0100" or "0111" or "1000" or "1011" or
 "1101" or "1100",

'0' when others;

$EVP \leq \text{not}(x)$;

$ODP \leq x$;

end Arch;

Exercice 5 :

Architecture Arch of parking is

Signal X : std_logic_vector (3 downto 0)

begin

process (clk, rst)

begin

if (rst = '1') then X <= "0000"

elsif (clk'event and clk = '1') then

if (up = '1') then X <= X + 1;

elsif (down = '1') then X <= X - 1;

endif;

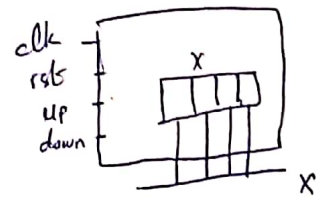
endif;

end process;

PD <= '1' when X < Max else '0';

PC <= '1' when X >= Max else '0';

end Arch;



(data flow :
when we have
equations

structural :
when we have
déjà des files VHDL)

Description structurelle

Entity CopDec is port (clk, rst, up, down : in std_logic;
X : out std_logic); end;

Arch:

Entity Comparateur is port (X, Nmax : in std_logic_vector (3→0)
PD, PC : out std_logic); end;

we have
this

⇒ Solution:

Entity park is port (clk, rst, up, down; in std_logic;
Nmax : in std_logic (3→0);
PD, PC : out std_logic); end;

Architecture Arch of park is

Component Cop-dec port (...); and Component;

Component Comparateur port (...); end component;

Signal Y : std_logic_vector (3 downto 0);

begin

Cop-dec port map (clk <= clk, rst <= rst,

Up <= up, down <= down, X <= Y);

Comparateur port map (X <= Y, Nmax <=

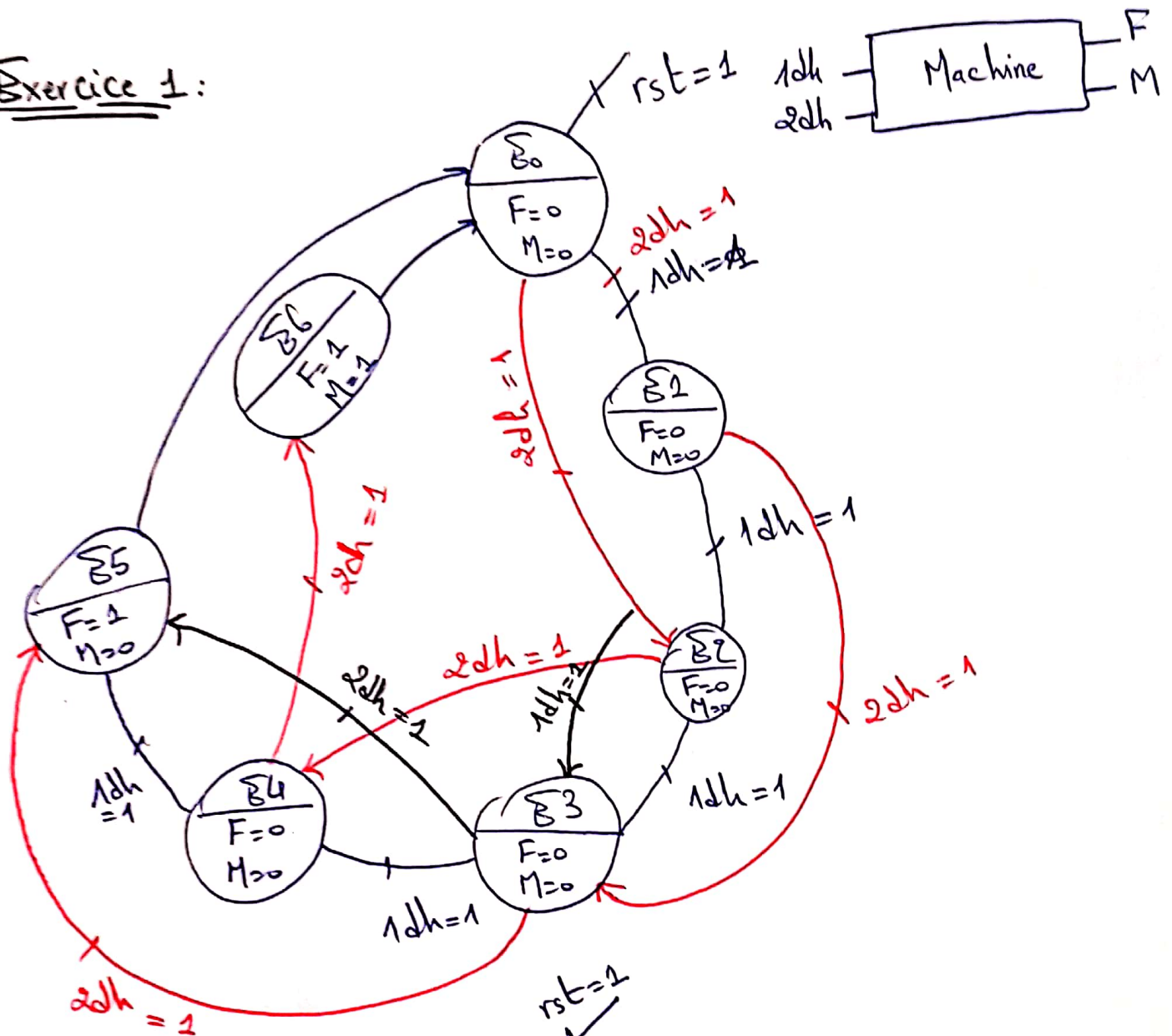
Nmax, PD <= PD, PC <= PC);

end Arch;

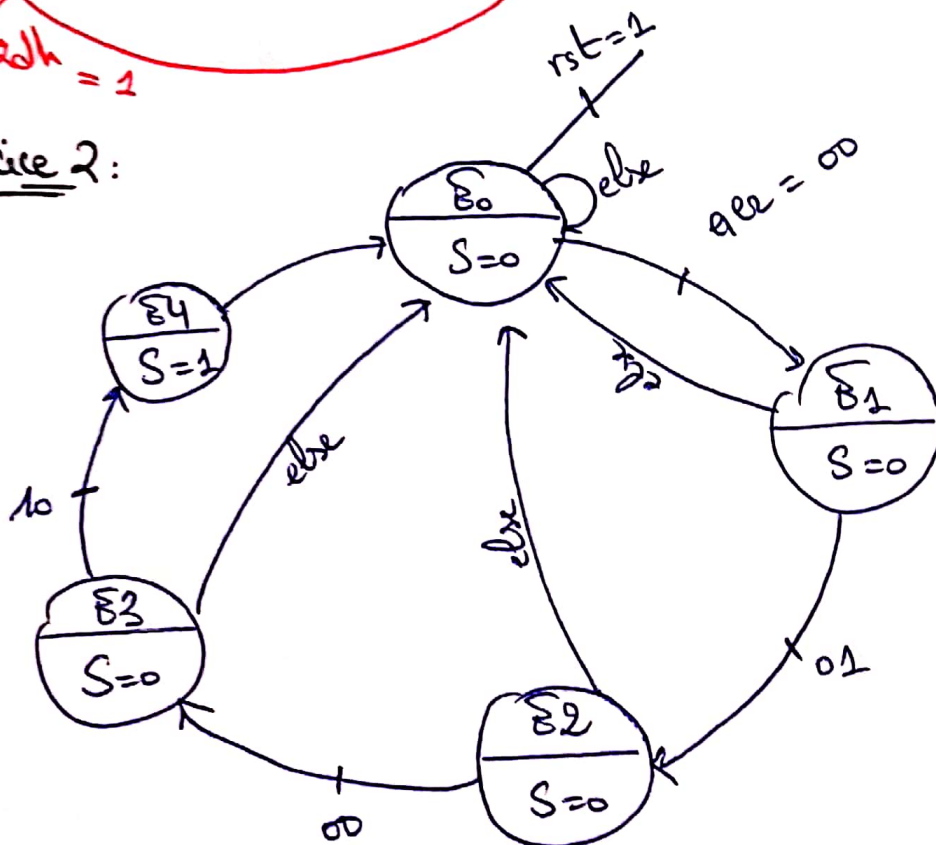


Td3

Exercise 1:



Exercise 2:



Exercise 3:

